Data Structures & Algorithms

A Basic Cheat Sheet

1. What is DSA?

A **data structure** is a way of organizing and storing data so it can be used efficiently. An **algorithm** is a step-by-step procedure for solving a problem. Choosing the right combination determines how fast and how memory-friendly your program is.

2. Time & Space Complexity (Big-O)

Big-O describes how running time or memory grows as the input size **n** grows, in the worst case.

Notation	Name	Example
O(1)	Constant	Access array element by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Scan a list once
O(n log n)	Linearithmic	Merge sort, quick sort (avg)
$O(n^2)$	Quadratic	Nested loops, bubble sort
$O(2^n)$	Exponential	Naive recursive Fibonacci

3. Arrays

A contiguous block of memory holding elements of the same type. Index access is O(1); searching is O(n); inserting or deleting in the middle is O(n) because elements must be shifted.

```
arr = [10, 20, 30, 40]
print(arr[2])        # 30  -> O(1)
arr.append(50)       # add at end -> O(1) amortized
arr.insert(1, 15)    # add in middle -> O(n)
```

4. Linked Lists

A chain of nodes where each node stores a value and a pointer to the next node. Insertion or deletion at a known position is O(1), but access by index is O(n). Variants: singly, doubly, and circular linked lists.

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

head = Node(1)
head.next = Node(2)
head.next.next = Node(3)   # 1 -> 2 -> 3
```

5. Stacks

LIFO (Last In, First Out). Main operations: *push*, *pop*, *peek*, all O(1). Used for undo features, function call stacks, balanced-parentheses checks, and DFS.

```
stack = []
stack.append(1)   # push
stack.append(2)
stack.pop()       # returns 2
```

6. Queues

FIFO (First In, First Out). Main operations: *enqueue* and *dequeue*, both O(1) with a proper implementation. Used in scheduling, buffering, and BFS. A **deque** allows insertion/removal at both ends.

```
from collections import deque
q = deque()
q.append(1)      # enqueue
q.append(2)
q.popleft()      # dequeue -> 1
```

7. Hash Tables (Dictionaries)

Store key-value pairs. A hash function maps each key to an index, giving average O(1) insert, lookup, and delete. Collisions (two keys mapping to the same index) are handled by chaining or open addressing. Great for counting frequencies, caching, and finding duplicates.

```
freq = {}
for ch in 'banana':
    freq[ch] = freq.get(ch, 0) + 1
# {'b': 1, 'a': 3, 'n': 2}
```

8. Trees

A hierarchical structure with a **root** and child nodes. A **binary tree** has at most two children per node. A **Binary Search Tree (BST)** keeps left values smaller and right values larger than the node, giving O(log n) search when balanced. Traversals: **inorder** (left, root, right), **preorder**, **postorder**, and **level-order** (BFS). Heaps are trees used to build priority queues.

```
class TreeNode:
    def __init__(self, val):
        self.val, self.left, self.right = val, None, None

def inorder(node):
    if node:
        inorder(node.left)
        print(node.val)
        inorder(node.right)
```

9. Graphs

A set of **vertices** (nodes) connected by **edges**. Graphs can be directed or undirected, weighted or unweighted. Common representations: adjacency list (space-efficient, preferred) and adjacency matrix. Core traversals: **BFS** (uses a queue, finds shortest path in unweighted graphs) and **DFS** (uses a stack or recursion).

```
graph = {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': []}

def bfs(start):
    seen, q = {start}, deque([start])
    while q:
        node = q.popleft()
        print(node)
        for nb in graph[node]:
            if nb not in seen:
                seen.add(nb)
                q.append(nb)
```

10. Searching Algorithms

Linear search checks each element in turn: O(n). **Binary search** repeatedly halves a *sorted* array: O(log n).

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        if arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
```

11. Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Idea
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Swap adjacent out-of-order pairs
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Pick the minimum each pass
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Insert into sorted portion
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Divide, sort halves, merge
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	Partition around a pivot

12. Key Algorithm Techniques

Recursion: a function that calls itself on a smaller problem, with a base case to stop.

Two pointers / sliding window: use two indices to scan arrays or strings in O(n) instead of $O(n^2)$.

Divide and conquer: split the problem, solve the parts, combine results (merge sort, binary search).

Greedy: take the locally best choice at every step (e.g., activity selection, Dijkstra's algorithm).

Dynamic programming: store results of overlapping subproblems to avoid recomputation.

```
def fib(n, memo={}):
    if n <= 1: return n
    if n not in memo:
        memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

13. Quick Reference: Data Structure Operations

Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)*	O(1)*
Stack / Queue	O(n)	O(n)	O(1)	O(1)
Hash Table	N/A	O(1) avg	O(1) avg	O(1) avg
BST (balanced)	O(log n)	O(log n)	O(log n)	O(log n)

* when the position/node is already known.